

EDA Notebook

0. Setup Environment

```
In [1]: # DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

```
12.9/12.9 MB 89.9 MB/s eta 0:00:00
4.9/4.9 MB 99.9 MB/s eta 0:00:00
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
Mounted at /content/gdrive

You can now save your data files in: /content/gdrive/MyDrive/36106/assignment/AT3/data

Student Information

```
In [2]: # <Student to fill this section>
group_name = "36106-26AU-AT3-Group 24"
student_name = "Nana Ama Goldwater"
student_id = "26137455"
```

```
In [3]: # Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

group_name

36106-26AU-AT3-Group 24

```
In [4]: # Do not modify this code
print_tile(size="h1", key='student_name', value=student_name)
```

student_name

Nana Ama Goldwater

```
In [5]: # Do not modify this code
print_tile(size="h1", key='student_id', value=student_id)
```

student_id

26137455

0. Python Packages

0.a Install Additional Packages

If you are using additional packages, you need to install them here using the command: !
`pip install <package_name>`

```
In [6]: # <Student to fill this section>
```

0.b Import Packages

```
In [7]: import pandas as pd
import altair as alt
import re

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 200)
```

B. Data Understanding

```
In [8]: # Do not modify this code
try:
    df = pd.read_csv(at.folder_path / "customer.csv")
except Exception as e:
    print(e)
```

[Errno 2] No such file or directory: '/content/gdrive/MyDrive/36106/assignment/AT3/data/customer.csv'

B.1 Explore Dataset

```
In [9]: # Structural overview: shape, dtypes, head/tail, missing values, duplicates.
df = pd.read_csv("/content/customer.csv")
print(f'Dataset shape : {df.shape[0]:,} rows x {df.shape[1]} columns\n')
df.info()

print('\nFirst 5 rows:')
display(df.head())
print('\nLast 5 rows:')
display(df.tail())
```

Dataset shape : 14,275 rows x 5 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14275 entries, 0 to 14274
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customer_id      14275 non-null  object
1   person_id        14038 non-null  object
2   store_id         452 non-null   object
3   territory_id     14275 non-null  object
4   account_number   14275 non-null  object
dtypes: object(5)
memory usage: 557.7+ KB
```

First 5 rows:

	customer_id	person_id	store_id	territory_id	account_number
0	4b5762b3-b7bf-4716-9bf9-5c1c9e1f375d	092707ff-4088-4416-ac86-14491f3120c2	NaN	0ad4c625-bb65-4376-8a41-0a65719b0db8	AW00019475
1	26bfba89-84b7-4b5b-9bab-caf7a2791006	fe043525-e0af-4578-a4b9-bfa7be28f4e4	NaN	25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	AW00022390
2	cf2cf4cc-fcce-4fe5-9400-78373420b525	ee25bbaf-5cdd-4ed4-92fe-7136c4e803a9	NaN	0ad4c625-bb65-4376-8a41-0a65719b0db8	AW00016320
3	83f558b8-bdab-4781-b112-55d2c275ff52	f0452f05-3690-4170-9fb0-fbf9995bbea7	NaN	2ac923e9-3043-4f5e-bae0-ad28089bf187	AW00026149
4	5edc53f4-7821-4a9b-9c27-a45dd2af713e	16c2532d-adb6-4a33-b0ea-71b87d83a904	NaN	0ad4c625-bb65-4376-8a41-0a65719b0db8	AW00021849

Last 5 rows:

	customer_id	person_id	store_id	territory_id	account_number
14270	ed815eee-e5f0-4e03-b706-4ce8d1a2dd7c	55d19600-3a17-4e4b-8863-b96427852de1	NaN	25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	AW00028374
14271	648057a9-519f-4d05-b64d-bdc2faf4f0e9	27f522c2-06f4-437f-ab39-b92e6969c844	NaN	e0e3bac4-790e-4617-84b3-be0c2bdb7070	AW00014784
14272	c238748a-f4f8-49e9-b462-330d2f7b68d5	0418b76b-eeed-482c-b086-796ee1bcc2b4	NaN	e0e3bac4-790e-4617-84b3-be0c2bdb7070	AW00016294
14273	ceb62a8b-d407-47b1-a20f-40e0d6c8944b	5d8610e3-4705-4f84-abab-ab3521f105aa	NaN	e0e3bac4-790e-4617-84b3-be0c2bdb7070	AW00016321
14274	c1198f86-cf66-4bc1-8e7d-79b40c1db804	cd70a799-a1a2-43a3-b2dc-03b2bce437ed	NaN	0ad4c625-bb65-4376-8a41-0a65719b0db8	AW00019255

```
In [10]: # Missing values, duplicates, and per-column uniqueness.
print('Missing values per column:')
miss = df.isna().sum().to_frame('n_missing')
miss['pct_missing'] = (df.isna().mean() * 100).round(2)
display(miss)

print(f'\nExact duplicate rows      : {df.duplicated().sum()}')
print(f'Unique customer_id values    : {df["customer_id"].nunique():,} / {len(df):,}')
print(f'Unique account_number values  : {df["account_number"].nunique():,} / {len(df):,}')
```

Missing values per column:

	n_missing	pct_missing
customer_id	0	0.00
person_id	237	1.66
store_id	13823	96.83
territory_id	0	0.00
account_number	0	0.00

```
Exact duplicate rows      : 3944
Unique customer_id values : 10,331 / 14,275
Unique account_number values : 9,392 / 14,275
```

```
In [11]: dataset_insights = """
Dataset: 'customer.csv' has 14,275 rows x 5 columns. This is the customer dimension table.

Columns (all stored as strings):
- customer_id      : UUID. Intended as primary key (see B.2 for caveat).
- person_id       : FK to person dimension; populated for INDIVIDUAL customers.
- store_id        : FK to store dimension; populated for BUSINESS customers.
- territory_id    : FK to sales territory dimension.
- account_number  : human-readable account code (e.g. 'AW00019475').
```

```

Quality / completeness:
- DATA QUALITY ISSUE: 3,944 exact-duplicate rows. Only 10,331 unique customer_id values across 14,275 rows. This must be deduplicated before any aggregation.(the prep notebook handles this in B.2 via drop_duplicates on customer_id).
- DATA QUALITY ISSUE: account_number is NOT one-to-one with customer_id in the raw file, 939 account numbers appear against multiple customer_id rows (see B.2). After dedup the relationship is one-to-one as expected.
- customer_id, territory_id and account_number are otherwise fully populated.
- person_id is missing for 237 rows (1.66%) and store_id for 13,823 rows (96.83%). These are NOT data errors, they are STRUCTURAL NULLs that encode the customer type (individual vs business).

Implications for the repeat-purchase use case:
- The table is small enough to load entirely into memory and merge.
- Deduplication is non-negotiable. Without it, every customer-level aggregate (RFM, recent activity, etc.) inflates by the duplication multiplier for affected customers.
- All IDs are UUIDs; they carry no business meaning and cannot be used as model features directly.
- The dominant business segment by row-count is individuals (~97%), so global model performance will be largely driven by the individual sub-population unless explicitly stratified.
"""

```

```

In [12]: # Do not modify this code
print_tile(size="h3", key='dataset_insights', value=dataset_insights)

```

dataset_insights

Dataset: 'customer.csv' has 14,275 rows x 5 columns. This is the customer dimension table. Columns (all stored as strings): - customer_id : UUID. Intended as primary key (see B.2 for caveat). - person_id : FK to person dimension; populated for INDIVIDUAL customers. - store_id : FK to store dimension; populated for BUSINESS customers. - territory_id : FK to sales territory dimension. - account_number : human-readable account code (e.g. 'AW00019475'). Quality / completeness: - DATA QUALITY ISSUE: 3,944 exact-duplicate rows. Only 10,331 unique customer_id values across 14,275 rows. This must be deduplicated before any aggregation.(the prep notebook handles this in B.2 via drop_duplicates on customer_id). - DATA QUALITY ISSUE: account_number is NOT one-to-one with customer_id in the raw file, 939 account numbers appear against multiple customer_id rows (see B.2). After dedup the relationship is one-to-one as expected. - customer_id, territory_id and account_number are otherwise fully populated. - person_id is missing for 237 rows (1.66%) and store_id for 13,823 rows (96.83%). These are NOT data errors, they are STRUCTURAL NULLs that encode the customer type (individual vs business). Implications for the repeat-purchase use case: - The table is small enough to load entirely into memory and merge. - Deduplication is non-negotiable. Without it, every customer-level aggregate (RFM, recent activity, etc.) inflates by the duplication multiplier for affected customers. - All IDs are UUIDs; they carry no business meaning and cannot be used as model features directly. - The dominant business segment by row-count is individuals (~97%), so global model performance will be largely driven by the individual sub-population unless explicitly stratified.

B.2 Explore Feature of Interest customer_id

```

In [13]: # customer_id is the table's primary key. Validate format and uniqueness.
uuid_re = re.compile(r'^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$')

is_valid = df['customer_id'].astype(str).str.match(uuid_re)
lengths = df['customer_id'].astype(str).str.len()

print(f'Total customer_id values : {len(df):,}')
print(f'Unique customer_id values : {df["customer_id"].nunique():,}')
print(f'Missing customer_id values : {df["customer_id"].isna().sum():,}')
print(f'Proportion matching UUID v-format: {is_valid.mean():.0%}')
print(f'String length stats:')

```

```
print(lengths.describe().round(2))
```

```
Total customer_id values : 14,275
Unique customer_id values : 10,331
Missing customer_id values : 0
Proportion matching UUID v-format: 100%
String length stats:
count    14275.0
mean      36.0
std        0.0
min       36.0
25%       36.0
50%       36.0
75%       36.0
max       36.0
Name: customer_id, dtype: float64
```

```
In [14]: # Confirm one-to-one alignment between customer_id and account_number.
# If they are perfectly aligned, account_number is redundant for joins.
n_pairs = df.groupby('customer_id')['account_number'].nunique()
print(f'Customers with >1 account_number : {(n_pairs > 1).sum()}')

rev_pairs = df.groupby('account_number')['customer_id'].nunique()
print(f'Account_numbers with >1 customer_id : {(rev_pairs > 1).sum()}')
```

```
Customers with >1 account_number : 0
Account_numbers with >1 customer_id : 939
```

```
In [15]: feature_1_insights = """
'customer_id' is the intended primary key of the customer table.

Distribution / format:
- 14,275 total values, 10,331 unique (72.4%). 3,944 customer_id values
  appear in more than one row – these are the exact-duplicate rows
  surfaced in B.1.
- Every value matches the canonical UUID v-format (8-4-4-4-12 hex with
  hyphens, length 36). No malformed IDs.
- Per-customer: each customer_id has exactly one account_number (0
  violations). The reverse holds in the OPPOSITE direction for 939
  account_numbers, which means those account_numbers appear in multiple
  DUPLICATE customer rows. After deduplication on customer_id the
  relationship becomes one-to-one in both directions, and account_number
  is then informationally redundant with customer_id.

Limitations / role:
- As a UUID, customer_id has no inherent ordering or meaning. It is
  suitable only as a join key, not as a model feature but a model that
  used the raw UUID would memorise customers rather than generalise.
- Duplication must be resolved before joining onto the sales fact
  table, or each duplicated customer would contribute their order
  history multiple times. The prep notebook deduplicates on customer_id
  with keep='first' in section B.2.
- Downstream prep code uses customer_id ONLY to join sales_order_header
  to customer attributes and to build the repeat-purchase label; it is
  dropped from the feature matrix before modelling.
"""
```

```
In [16]: # Do not modify this code
print_tile(size="h3", key='feature_1_insights', value=feature_1_insights)
```

'customer_id' is the intended primary key of the customer table.
 Distribution / format: - 14,275 total values, 10,331 unique (72.4%). 3,944 customer_id values appear in more than one row — these are the exact-duplicate rows surfaced in B.1. - Every value matches the canonical UUID v-format (8-4-4-4-12 hex with hyphens, length 36). No malformed IDs. - Per-customer: each customer_id has exactly one account number (0 violations). The reverse holds in the OPPOSITE direction for 939 account_numbers, which means those account_numbers appear in multiple DUPLICATE customer rows. After deduplication on customer_id the relationship becomes one-to-one in both directions, and account_number is then informationally redundant with customer_id.
 Limitations / role: - As a UUID, customer_id has no inherent ordering or meaning. It is suitable only as a join key, not as a model feature but a model that used the raw UUID would memorise customers rather than generalise. - Duplication must be resolved before joining onto the sales fact table, or each duplicated customer would contribute their order history multiple times. The prep notebook deduplicates on customer_id with keep = 'first' in section B.2. - Downstream prep code uses customer_id ONLY to join sales_order_header to customer attributes and to build the repeat-purchase label; it is dropped from the feature matrix before modelling.

B.3 Explore Feature of Interest `person_id` and `store_id` (customer type)

```
In [17]: # person_id is populated for individual customers; store_id for businesses.
# Cross-tabulate the NULL patterns to confirm the structural meaning of NULLs.
has_person = df['person_id'].notna()
has_store   = df['store_id'].notna()

type_mix = pd.crosstab([has_person.rename('has_person_id'),
                        has_store.rename('has_store_id'),
                        margins=True, margins_name='total'])

display(type_mix)

individuals = (has_person & ~has_store).sum()
businesses  = (~has_person & has_store).sum()
both        = (has_person & has_store).sum()
neither     = (~has_person & ~has_store).sum()

print(f'\nIndividual customers (person only) : {individuals:>5,} ({individuals/len(df):.2%}')
```

has_store_id	False	True	total
has_person_id			
False	0	237	237
True	13823	215	14038
total	13823	452	14275

```
Individual customers (person only) : 13,823 (96.83%)
Business customers (store only)   :    237 (1.66%)
Both person AND store present     :    215 (1.51%)
Neither person NOR store present  :      0 (0.00%)
```

```
In [18]: # Visualise the customer-type split (the derived feature most relevant
# to the repeat-purchase model).
import pandas as pd
type_df = pd.DataFrame({
    'customer_type': ['individual (person only)', 'business (store only)',
                     'both present', 'neither present'],
    'count'         : [individuals, businesses, both, neither],
})
```

```
alt.Chart(type_df).mark_bar().encode(
    x=alt.X('count:Q', title='Number of customers'),
    y=alt.Y('customer_type:N', sort='-x', title=None),
    tooltip=['customer_type', 'count']
).properties(width=500, height=180,
              title='Customer type breakdown (derived from person_id / store_id NULLs)')
```

Out[18]:

```
In [19]: feature_2_insights = """
`person_id` and `store_id` are STRUCTURALLY NULL columns whose NULL/non-NULL
pattern encodes the customer type.

Joint distribution (before deduplication of customer_id):
- Individual customers (person_id present, store_id NULL): 13,823 (96.83%).
- Business customers (store_id present, person_id NULL): 237 (1.66%).
- Customers with BOTH columns populated : 215 (1.51%).
- Customers with NEITHER column populated : 0.

Data-quality findings:
- These NULLs are NOT missing data. They CARRY information ('this customer
is an individual / a business / both') and must NOT be imputed away.
- 215 customers have BOTH person_id AND store_id populated. This appears
to be a legitimate (small) segment likely sole-trader / owner-operator
accounts where the same entity is both an individual person and a
business store. The prep notebook treats `is_individual` and
`is_business` as two SEPARATE binary flags rather than a single
mutually-exclusive enum, so this 1.5% segment can be modelled
explicitly.

Implications for repeat-purchase modelling:
- The dataset is heavily skewed toward individual customers (~97%).
A global model will predominantly learn individual behaviour.
- The 237 store-only and 215 dual-flag customers, while only ~3.2% of
rows, may contribute a much larger share of revenue (one B2B order
can equal hundreds of consumer orders confirmed in the
sales_order_header EDA's bimodal money distribution). Whether they
should be weighted differently in the loss function depends on the
downstream business metric the model is optimised for.
"""
```

```
In [20]: # Do not modify this code
print_tile(size="h3", key='feature_2_insights', value=feature_2_insights)
```

feature_2_insights

`person\_id` and `store\_id` are STRUCTURALLY NULL columns whose NULL/non-NULL pattern encodes the customer type. Joint distribution (before deduplication of customer_id): - Individual customers (person_id present, store_id NULL): 13,823 (96.83%). - Business customers (store_id present, person_id NULL): 237 (1.66%). - Customers with BOTH columns populated : 215 (1.51%). - Customers with NEITHER column populated : 0. Data-quality findings: - These NULLs are NOT missing data. They CARRY information ('this customer is an individual / a business / both') and must NOT be imputed away. - 215 customers have BOTH person_id AND store_id populated. This appears to be a legitimate (small) segment likely sole-trader / owner-operator accounts where the same entity is both an individual person and a business store. The prep notebook treats `is\_individual` and `is\_business` as two SEPARATE binary flags rather than a single mutually-exclusive enum, so this 1.5% segment can be modelled explicitly. Implications for repeat-purchase modelling: - The dataset is heavily skewed toward individual customers (~97%). A global model will predominantly learn individual behaviour. - The 237 store-only and 215 dual-flag customers, while only ~3.2% of rows, may contribute a much larger share of revenue (one B2B order can equal hundreds of consumer orders confirmed in the sales_order_header EDA's bimodal money distribution). Whether they should be weighted differently in the loss function depends on the downstream business metric the model is optimised for.

B.4 Explore Feature of Interest `territory_id`

```
In [21]: # Customers are distributed across a small number of territories. Each
# territory is identified by a UUID, so we count and rank them by row share.
ter_counts = df['territory_id'].value_counts().reset_index()
ter_counts.columns = ['territory_id', 'n_customers']
ter_counts['short_id'] = ter_counts['territory_id'].str[:8]
ter_counts['pct_share'] = ter_counts['n_customers'] / len(df)

print(f'Number of distinct territories : {ter_counts.shape[0]}')
print(f'Top-1 territory share          : {ter_counts["pct_share"].iloc[0]:.2%}')
print(f'Top-3 territories combined      : {ter_counts["pct_share"].head(3).sum():.2%}')
display(ter_counts.head(10).style.format({'pct_share': '{:.2%}'})
```

```
Number of distinct territories : 4
Top-1 territory share          : 39.00%
Top-3 territories combined      : 80.04%
```

	territory_id	n_customers	short_id	pct_share
0	2ac923e9-3043-4f5e-bae0-ad28089bf187	5567	2ac923e9	39.00%
1	e0e3bac4-790e-4617-84b3-be0c2bdb7070	3008	e0e3bac4	21.07%
2	25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	2851	25d73ad2	19.97%
3	0ad4c625-bb65-4376-8a41-0a65719b0db8	2849	0ad4c625	19.96%

```
In [22]: # Bar chart of customer counts per territory, sorted descending.
alt.Chart(ter_counts).mark_bar().encode(
    x=alt.X('n_customers:Q', title='Number of customers'),
    y=alt.Y('short_id:N', sort='-x', title='Territory (first 8 chars of UUID)'),
    tooltip=['territory_id', 'n_customers',
             alt.Tooltip('pct_share:Q', format='.2%')]
).properties(width=480, height=260,
             title='Customer distribution across territories')
```

Out[22]:

```
In [23]: feature_n_insights = """
`territory_id` is the geography dimension key for each customer.

Distribution:
- Fully populated, no missing values.
- Only 4 distinct territories, substantially lower cardinality than a
  typical AdventureWorks deployment (which carries 10 territories at
  sales-territory level). The retailer in this dataset is concentrated
  in a small number of regions.
- Distribution is moderately concentrated: the top territory holds
  ~39% of customers and the top three combined hold ~80%. The 4th
  territory is a long-tail with ~20% of customers spread thinly.
- Low cardinality makes territory_id a strong candidate for one-hot
  encoding (handled in the prep notebook's Section E.3). After
  one-hot encoding, territory contributes only 4 columns to the model.

Implications for repeat-purchase modelling:
- Territory captures regional differences in customer behaviour
  (purchasing power, channel mix, seasonality). It is retained as a
  model feature, not dropped.
- With only 4 territories the model's geographic-effect coefficients
  will be relatively stable (vs a long-tailed territory column where
  rare territories would be under-supported and noisy).
- The sales_order_header table also carries a territory_id (the
  territory of the order, not necessarily the customer's home
  territory). Reconciling the two is handled in prep B.4: if an order's
  territory_id is missing it is back-filled from the customer dimension.

Caveat:
- Encoding territory as a UUID hides any natural geographic adjacency.
  A model cannot tell that two neighbouring territories are 'similar'.
  If geographic clustering matters downstream, a richer geography feature
  (continent, country) would be needed out of scope here.
"""
```

```
In [24]: # Do not modify this code
print_tile(size="h3", key='feature_n_insights', value=feature_n_insights)
```


`territory\_id` is the geography dimension key for each customer. Distribution: - Fully populated, no missing values. - Only 4 distinct territories, substantially lower cardinality than a typical AdventureWorks deployment (which carries 10 territories at sales-territory level). The retailer in this dataset is concentrated in a small number of regions. - Distribution is moderately concentrated: the top territory holds ~39% of customers and the top three combined hold ~80%. The 4th territory is a long-tail with ~20% of customers spread thinly. - Low cardinality makes territory_id a strong candidate for one-hot encoding (handled in the prep notebook's Section E.3). After one-hot encoding, territory contributes only 4 columns to the model. Implications for repeat-purchase modelling: - Territory captures regional differences in customer behaviour (purchasing power, channel mix, seasonality). It is retained as a model feature, not dropped. - With only 4 territories the model's geographic-effect coefficients will be relatively stable (vs a long-tailed territory column where rare territories would be under-supported and noisy). - The sales_order_header table also carries a territory_id (the territory of the order, not necessarily the customer's home territory). Reconciling the two is handled in prep B.4: if an order's territory_id is missing it is back-filled from the customer dimension. Caveat: - Encoding territory as a UUID hides any natural geographic adjacency. A model cannot tell that two neighbouring territories are 'similar'. If geographic clustering matters downstream, a richer geography feature (continent, country) would be needed out of scope here.

B.5 Ethics & Privacy considerations

```
In [27]: ethics_privacy_insights = """
Ethics & Privacy considerations
customer dimension

PII / re-identification risk:
- Although customer_id, person_id, store_id and account_number are UUIDs
  rather than names, they are still personal information under the
  Australian Privacy Act 1988 (Cth) once they can be linked back to a
  named customer record. APP 6 (use and disclosure) and APP 11 (security)
  therefore apply to any downstream pipeline that consumes this table.
- The 215 customers with BOTH person_id and store_id populated represent
  a small (~1.5%) segment of sole-traders / owner-operators. Small
  segments carry higher re-identification risk because they are unique
  or near-unique in combination with other attributes.

Indigenous data considerations:
- This dimension carries no explicit ethnicity, language or community
  attributes, so it does not directly process Indigenous data. However,
  if any portion of the customer base includes Aboriginal or Torres
  Strait Islander peoples, the AIATSIS Code of Ethics (2020) and the
  CARE principles (Collective benefit, Authority to control,
  Responsibility, Ethics) apply to any downstream use of derived
  features. We do NOT use customer_id, person_id or store_id as model
  features; they are dropped before modelling in the prep notebook.

Mitigations already applied:
- UUIDs only, no names, emails, addresses or contact details enter the
  EDA or feature matrix.
- Structural NULLs (person_id / store_id) are converted to non-personal
  binary flags rather than retained as raw identifiers.

Recommendations for downstream use:
- Enforce role-based access on the customer table; log every read.
- Honour customer erasure requests in any retraining cache, not just
  in production tables.
- Re-check uniqueness of small segments (<2% of rows) before publishing
  any aggregate that could re-identify them.
"""
print_tile(size="h3", key='ethics_privacy_insights', value=ethics_privacy_insights)
```

Ethics & Privacy considerations customer dimension PII / re-identification risk: - Although customer_id, person_id, store_id and account_number are UUIDs rather than names, they are still personal information under the Australian Privacy Act 1988 (Cth) once they can be linked back to a named customer record. APP 6 (use and disclosure) and APP 11 (security) therefore apply to any downstream pipeline that consumes this table. - The 215 customers with BOTH person_id and store_id populated represent a small (~1.5%) segment of sole-traders / owner-operators. Small segments carry higher re-identification risk because they are unique or near-unique in combination with other attributes. Indigenous data considerations: - This dimension carries no explicit ethnicity, language or community attributes, so it does not directly process Indigenous data. However, if any portion of the customer base includes Aboriginal or Torres Strait Islander peoples, the AIATSIS Code of Ethics (2020) and the CARE principles (Collective benefit, Authority to control, Responsibility, Ethics) apply to any downstream use of derived features. We do NOT use customer_id, person_id or store_id as model features; they are dropped before modelling in the prep notebook. Mitigations already applied: - UUIDs only, no names, emails, addresses or contact details enter the EDA or feature matrix. - Structural NULLs (person_id / store_id) are converted to non-personal binary flags rather than retained as raw identifiers. Recommendations for downstream use: - Enforce role-based access on the customer table; log every read. - Honour customer erasure requests in any retraining cache, not just in production tables. - Re-check uniqueness of small segments (2% of rows) before publishing any aggregate that could re-identify them.